
BOOK 6 — JAVA I/O & JAVA 8

Complete Fresher / Entry-Level Java Full Stack Preparation Series

CHAPTER 1 — JAVA I/O & FILE HANDLING

1.1 File Handling Fundamentals & Stream Architecture

In Java, **Input/Output (I/O)** refers to the process of reading data from an input source (such as a file, keyboard, or network socket) and writing data to an output destination (such as a file, console, or network stream).

- **Stream Concept:** A **Stream** in Java I/O represents a continuous sequence of data flowing from a source to a destination.
 - **Input Stream:** Reads data from a source into a Java application.
 - **Output Stream:** Writes data from a Java application to a destination.
 - **Byte Streams vs Character Streams:**
 - **Byte Streams (`InputStream` / `OutputStream`):** Handle raw 8-bit bytes of binary data. Ideal for images, audio files, compiled bytecode, and raw data files.
 - **Character Streams (`Reader` / `Writer`):** Handle 16-bit Unicode characters. Automatically handle character encoding/decoding, making them ideal for plain text files (`.txt`, `.csv`, `.md`).
-

1.2 `java.io.File` Class

The `File` class is an abstract, platform-independent representation of file and directory pathnames. An instance of `File` does NOT represent the actual contents of a file; rather, it represents the file's path and metadata.

Key `File` Class Methods

- `createNewFile()`: Creates a new empty file if it does not already exist (returns `boolean`).
 - `mkdir()` / `mkdirs()`: Creates directories along the specified path.
 - `exists()`: Checks whether the file or directory exists.
 - `isFile()` / `isDirectory()`: Checks if the path refers to a file or a directory.
 - `getName()`: Returns the name of the file or directory.
 - `getAbsolutePath()`: Returns the absolute path string.
 - `length()`: Returns the size of the file in bytes (as a `long`).
 - `delete()`: Deletes the file or directory.
-

1.3 Character Streams: `FileReader` & `FileWriter`

`FileReader` and `FileWriter` are concrete character stream classes designed for reading and writing text files using character encodings. Charset behavior depends on the constructor used and the Java version (for example, historical default platform encoding vs. explicit or UTF-8 constructors in newer Java versions), and an explicit `Charset` can be used when required.

`FileWriter`

Used to write characters to a text file.

- **Constructors:**

- `FileWriter(File file)` / `FileWriter(String fileName)`: Overwrites existing content.
- `FileWriter(String fileName, boolean append)`: Appends data to existing content when `append` is `true`.

`FileReader`

Used to read characters from a text file.

- Reads data character-by-character (returns character ASCII/Unicode value as `int`, or `-1` when the end of file - EOF - is reached).

1.4 Buffered Character Streams: `BufferedReader` & `BufferedWriter`

Unbuffered character streams (`FileReader`/`FileWriter`) read or write directly from/to disk for every character, resulting in high I/O latency. **Buffered streams** wrapper around underlying readers/writers to buffer data in memory (where the buffer size is implementation/API dependent, with many implementations using a default buffer such as 8KB), dramatically improving read/write efficiency.

Key Advantages of `BufferedReader` & `BufferedWriter`

- `BufferedReader.readLine()`: Reads an entire line of text as a `String` (returns `null` at EOF), eliminating character-by-character reading logic.
- `BufferedWriter.newLine()`: Writes a platform-dependent line separator (`\n` on Linux/Mac, `\r\n` on Windows).
- `BufferedWriter.flush()`: Writes any buffered output characters to the underlying destination immediately.
- `BufferedWriter.close()`: Closes the writer, releasing associated resources, and automatically performs the required flushing prior to closing.

1.5 Object Serialization & Deserialization

Serialization is the process of converting the state of a Java object into a byte stream so it can be saved to a file, sent over a network, or stored in a database. **Deserialization** is the reverse process of converting a byte stream back into a live Java object in Heap memory.

Serialization Rules

- **`java.io.Serializable`**: A class MUST implement the `Serializable` marker interface to permit its instances to be serialized.
 - **`transient` Keyword**: Transient instance fields are not serialized by Java's default serialization mechanism and receive default primitive or null values after normal deserialization unless custom logic restores them.
 - **`serialVersionUID`**: A 64-bit version identifier that helps verify serialization compatibility between the serialized class (sender) and the current class definition during deserialization, ensuring compatible classes are loaded.
 - **Classes Used**: `ObjectOutputStream.writeObject(Object obj)` for writing, and `ObjectInputStream.readObject()` for reading.
-

1.6 Basic I/O Coding Problems (Syllabus 23.6.1 to 23.6.5)

Problem 1: Create a File (23.6.1)

```
import java.io.File;
import java.io.IOException;

public class CreateFileDemo {
    public static void main(String[] args) {
        File file = new File("sample.txt");

        try {
            // Attempting to create new empty file on disk
            if (file.createNewFile()) {
                System.out.println("File created successfully: " + file.getName());
                System.out.println("Absolute Path: " + file.getAbsolutePath());
            } else {
                System.out.println("File already exists at: " + file.getAbsolutePath());
            }
        } catch (IOException e) {
            System.out.println("An I/O error occurred during file creation: " + e.getMessage());
        }
    }
}
```

• **Expected Output:**

```
File created successfully: sample.txt
Absolute Path: /workspace/scratch/sample.txt
```

Problem 2: Write Data to a File (23.6.2)

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class WriteFileDemo {
    public static void main(String[] args) {
        String filePath = "sample.txt";
```

```
// Using try-with-resources for automatic resource cleanup
try (BufferedWriter writer = new BufferedWriter(new FileWriter(filePath))) {
    writer.write("Welcome to Java Full Stack Preparation!");
    writer.newLine(); // Platform-independent new line
    writer.write("Chapter 1: Java I/O and File Handling.");
    writer.newLine();
    writer.write("Writing text using BufferedWriter.");
    System.out.println("Data successfully written to " + filePath);
} catch (IOException e) {
    System.out.println("Error writing to file: " + e.getMessage());
}
}
```

• **Expected Output:**

```
Data successfully written to sample.txt
```

Problem 3: Read Data from a File (23.6.3)

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class ReadFileDemo {
    public static void main(String[] args) {
        String filePath = "sample.txt";

        try (BufferedReader reader = new BufferedReader(new FileReader(filePath))) {
            String line;
            System.out.println("--- File Contents ---");
            // readLine() returns null when EOF is reached
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            System.out.println("Error reading file: " + e.getMessage());
        }
    }
}
```

• **Expected Output:**

```
--- File Contents ---
Welcome to Java Full Stack Preparation!
Chapter 1: Java I/O and File Handling.
Writing text using BufferedWriter.
```

Problem 4: Copy File Content (23.6.4)

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
```

```

public class CopyFileDemo {
    public static void main(String[] args) {
        String sourcePath = "sample.txt";
        String destinationPath = "sample_copy.txt";

        try (BufferedReader reader = new BufferedReader(new FileReader(sourcePath));
            BufferedWriter writer = new BufferedWriter(new FileWriter(destinationPath))) {

            String line;
            while ((line = reader.readLine()) != null) {
                writer.write(line);
                writer.newLine();
            }
            System.out.println("File copied successfully from " + sourcePath + " to " + destinationPath);

        } catch (IOException e) {
            System.out.println("Error copying file: " + e.getMessage());
        }
    }
}

```

• **Expected Output:**

```
File copied successfully from sample.txt to sample_copy.txt
```

Problem 5: Count Lines and Words in a File (23.6.5)

```

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class FileWordCountDemo {
    public static void main(String[] args) {
        String filePath = "sample.txt";
        int lineCount = 0;
        int wordCount = 0;
        int charCount = 0;

        try (BufferedReader reader = new BufferedReader(new FileReader(filePath))) {
            String line;
            while ((line = reader.readLine()) != null) {
                lineCount++;
                charCount += line.length();

                // Splitting line into words based on whitespace regex
                String[] words = line.trim().split("\\s+");
                if (words.length > 0 && !words[0].isEmpty()) {
                    wordCount += words.length;
                }
            }

            System.out.println("--- File Statistics ---");
            System.out.println("Total Lines: " + lineCount);
            System.out.println("Total Words: " + wordCount);
            System.out.println("Total Characters (excluding line breaks): " + charCount);
        }
    }
}

```

```

        } catch (IOException e) {
            System.out.println("Error reading file statistics: " + e.getMessage());
        }
    }
}

```

• **Expected Output:**

```

--- File Statistics ---
Total Lines: 3
Total Words: 17
Total Characters (excluding line breaks): 112

```

CHAPTER 2 — JAVA 8 LAMBDA EXPRESSIONS & FUNCTIONAL INTERFACES

2.1 Lambda Expression Fundamentals & Syntax

Introduced in Java 8, a **Lambda Expression** is an anonymous (nameless) function that provides a clear and concise syntax to represent a Single Abstract Method (SAM) interface implementation directly as an expression.

Syntax Breakdown

```
(parameter_list) -> { lambda_body }
```

- **Parameter List:** Matches the arguments of the single abstract method. Type definitions are optional due to **Java Type Inference**.
- **Arrow Operator (`->`):** Separates parameters from the execution body.
- **Lambda Body:** Can be a single expression (implicit return) or a code block enclosed in `{}` (requires explicit `return` if non-void).

Variable Capture & "Effectively Final" Rule

Lambdas can access local variables from their enclosing outer scope **ONLY** if those variables are declared `final` or are **effectively final** (never modified after initialization).

2.2 Functional Interfaces (`@FunctionalInterface`)

A **Functional Interface** is an interface that contains **EXACTLY ONE abstract method** (Single Abstract Method - SAM). It can contain any number of `default` or `static` methods.

`@FunctionalInterface` Annotation

An optional annotation that instructs the Java compiler to enforce the SAM rule at compile time.

Core Built-In Functional Interfaces (`java.util.function`)

| Functional Interface | Abstract Method Signature | Input Parameter | Return Type | Common Use Case |

| :--- | :--- | :--- | :--- | :--- |

| **Predicate<T>** | `boolean test(T t)` | `T` | `boolean` | Conditional filtering (`filter()`) |

| **Function<T, R>** | `R apply(T t)` | `T` | `R` | Value transformation (`map()`) |

| **Consumer<T>** | `void accept(T t)` | `T` | `void` | Side-effects / Printing (`forEach()`) |

| **Supplier<T>** | `T get()` | `None` | `T` | Object generation / Lazy factory |

```
import java.util.function.Consumer;
import java.util.function.Function;
import java.util.function.Predicate;
import java.util.function.Supplier;

public class CoreFunctionalInterfacesDemo {
    public static void main(String[] args) {
        // 1. Predicate: Check if a number is even
        Predicate<Integer> isEven = num -> num % 2 == 0;
        System.out.println("Is 10 even? " + isEven.test(10)); // true

        // 2. Function: Convert String to its length
        Function<String, Integer> stringLength = str -> str.length();
        System.out.println("Length of 'Java 8': " + stringLength.apply("Java 8")); // 6

        // 3. Consumer: Print string in uppercase
        Consumer<String> printUpper = str -> System.out.println(str.toUpperCase());
        printUpper.accept("hello full stack developer");

        // 4. Supplier: Supply default string
        Supplier<Double> randomSupplier = () -> Math.random();
        System.out.println("Random Value: " + randomSupplier.get());
    }
}
```

2.3 Method References (::)

A **Method Reference** is a compact, readable shorthand syntax for a Lambda Expression that executes an existing named method or constructor.

Four Types of Method References

- **Reference to a Static Method:** `ClassName::staticMethodName`
 - Lambda: `str -> Integer.parseInt(str)`
 - Method Ref: `Integer::parseInt`
- **Reference to an Instance Method of a Particular Object:** `instanceRef::instanceMethodName`
 - Lambda: `str -> System.out.println(str)`
 - Method Ref: `System.out::println`
- **Reference to an Instance Method of an Arbitrary Object of a Particular Type:** `ClassName::instanceMethodName`
 - Lambda: `str -> str.toUpperCase()`

- Method Ref: ``String::toUpperCase``
 - **Reference to a Constructor:** ``ClassName::new``
 - Lambda: ``() -> new ArrayList<>()``
 - Method Ref: ``ArrayList::new``
-

CHAPTER 3 — JAVA 8 STREAM API, OPTIONAL & DATE/TIME API

3.1 Stream API Architecture & Lifecycle

Introduced in Java 8, the **Stream API** (``java.util.stream``) provides a functional-style, declarative framework for processing sequences of elements (collections, arrays, or I/O resources).

Streams vs Collections

- **Collections** are data structures focused on how data is stored and organized in memory.
 - **Streams** are computational pipelines focused on specifying what calculations to perform on data. Streams do NOT store data in memory.
 - **Non-Mutating Source Operations:** Stream operations generally do not modify the source collection itself (producing a new stream or terminal result), although code inside user-defined lambda expressions could potentially cause side effects if not designed carefully.
 - **Single-Use:** Once a terminal operation is executed on a stream, the stream pipeline is consumed and cannot be reused.
-

3.2 Intermediate Stream Operations (Lazy Evaluation)

Intermediate operations transform a stream into another stream. They are **lazy**; they accumulate pipeline operations and execute ONLY when a terminal operation is invoked.

Key Intermediate Operations

- ``filter(Predicate<T>)``: Evaluates elements against a boolean test and retains matching elements.
 - ``map(Function<T, R>)``: Applies a transformation function to each element, producing a stream of converted values.
 - ``sorted()`` / ``sorted(Comparator<T>)``: Sorts stream elements in natural order or according to a custom comparator.
 - ``distinct()``: Removes duplicate elements (based on ``equals()``).
 - ``limit(long maxSize)``: Truncates the stream to a maximum number of elements.
-

3.3 Terminal Stream Operations (Pipeline Execution)

Terminal operations consume the stream pipeline, trigger execution of all lazy intermediate operations, and return a non-stream result (such as a `List`, `long` count, or `Optional`).

Key Terminal Operations

- `forEach(Consumer<T>)`: Iterates over each stream element and performs an action.
- `collect(Collector)`: Accumulates stream elements into a target container (e.g., `Collectors.toList()` collects elements into a `List`, without guaranteeing a specific concrete `List` implementation; `Collectors.toSet()`, `Collectors.joining()`).
- `reduce(identity, accumulator)`: Combines stream elements into a single summary value using an associative operation.
- `count()`: Returns the total count of elements in the stream as a `long`.

3.4 `Optional` Class Basic Awareness

`java.util.Optional<T>` is a container object introduced in Java 8 that represents a value that may be present or empty, helping make absence handling explicit across API signatures.

Key `Optional` Methods

- `Optional.of(value)`: Returns an `Optional` containing the specified non-null value. Calling `Optional.of(null)` throws a `NullPointerException`.
- `Optional.ofNullable(value)`: Returns an `Optional` describing the specified value if non-null; if passed `null` (`Optional.ofNullable(null)`), returns an empty `Optional`.
- `Optional.empty()`: Returns an empty `Optional` instance.
- `isPresent()` / `isEmpty()`: Checks if a value is enclosed.
- `ifPresent(Consumer)`: Executes action if value is present.
- `orElse(fallbackValue)`: Returns enclosed value if present, or `fallbackValue` if empty.

```
import java.util.Optional;

public class OptionalDemo {
    public static void main(String[] args) {
        String name = "Alice";
        String nullName = null;

        Optional<String> optName = Optional.ofNullable(name);
        Optional<String> optNull = Optional.ofNullable(nullName);

        System.out.println("Name present? " + optName.isPresent()); // true
        optName.ifPresent(n -> System.out.println("User: " + n));

        // Using orElse to provide a safe default value
        String result = optNull.orElse("Default Guest");
        System.out.println("Result: " + result); // Default Guest
    }
}
```

3.5 New Date & Time API (`java.time`)

Prior to Java 8, legacy date handling (`java.util.Date`, `java.util.Calendar`, `SimpleDateFormat`) suffered from major flaws: mutable state, lack of thread safety, confusing zero-indexed months, and non-intuitive APIs. Java 8 introduced a modern Date & Time API inside `java.time`, featuring core classes that are immutable and thread-safe.

Core `java.time` Classes

- **`LocalDate`**: An immutable, thread-safe class representing a date (year, month, day) without time or time-zone (e.g., `2026-09-17`).
- **`LocalTime`**: An immutable, thread-safe class representing a time (hour, minute, second, nanosecond) without date or time-zone (e.g., `14:30:00`).
- **`LocalDateTime`**: An immutable, thread-safe class combining date and time (e.g., `2026-09-17T14:30:00`).
- **`DateTimeFormatter`**: Formats and parses dates in an immutable and thread-safe manner.

```
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;

public class DateTimeApiDemo {
    public static void main(String[] args) {
        LocalDate currentDate = LocalDate.now();
        LocalTime currentTime = LocalTime.now();
        LocalDateTime currentDateTime = LocalDateTime.now();

        System.out.println("Current Date: " + currentDate);
        System.out.println("Current Time: " + currentTime);
        System.out.println("Current DateTime: " + currentDateTime);

        // Formatting LocalDateTime
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss");
        String formattedDate = currentDateTime.format(formatter);
        System.out.println("Formatted Date: " + formattedDate);
    }
}
```

CHAPTER 4 — COMPLETE JAVA 8 CODING PRACTICE

Common Java 8 Coding Problems (Syllabus 24.13.1 to 24.13.6)

Problem 1: Filter Even Numbers Using Stream (24.13.1)

```
import java.util.Arrays;
import java.util.List;
```

```
import java.util.stream.Collectors;

public class StreamFilterEvenDemo {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

        // Filtering even numbers using Stream API
        List<Integer> evenNumbers = numbers.stream()
            .filter(n -> n % 2 == 0) // Predicate checking if number is even
            .collect(Collectors.toList());

        System.out.println("Original Numbers: " + numbers);
        System.out.println("Filtered Even Numbers: " + evenNumbers);
    }
}
```

• **Expected Output:**

```
Original Numbers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Filtered Even Numbers: [2, 4, 6, 8, 10]
```

Problem 2: Filter Numbers Greater Than a Value (24.13.2)

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class StreamFilterGreaterDemo {
    public static void main(String[] args) {
        List<Integer> scores = Arrays.asList(45, 78, 92, 31, 88, 64, 95);
        int threshold = 70;

        // Filtering numbers greater than threshold (70)
        List<Integer> highScores = scores.stream()
            .filter(score -> score > threshold)
            .collect(Collectors.toList());

        System.out.println("Scores above " + threshold + ": " + highScores);
    }
}
```

• **Expected Output:**

```
Scores above 70: [78, 92, 88, 95]
```

Problem 3: Convert List Using map() (24.13.3)

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class StreamMapDemo {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("java", "python", "spring", "microservices");

        // Transforming strings to uppercase using map()
        List<String> upperNames = names.stream()
            .map(String::toUpperCase)
            .collect(Collectors.toList());
    }
}
```

```

        .map(String::toUpperCase) // Method reference transformation
        .collect(Collectors.toList());

    System.out.println("Original Names: " + names);
    System.out.println("Uppercase Names: " + upperNames);
}
}

```

• **Expected Output:**

```

Original Names: [java, python, spring, microservices]
Uppercase Names: [JAVA, PYTHON, SPRING, MICROSERVICES]

```

Problem 4: Sort List Using Stream (24.13.4)

```

import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
import java.util.stream.Collectors;

public class StreamSortDemo {
    public static void main(String[] args) {
        List<String> fruits = Arrays.asList("Banana", "Apple", "Mango", "Cherry", "Date");

        // 1. Natural Ascending Sorting
        List<String> sortedAsc = fruits.stream()
            .sorted()
            .collect(Collectors.toList());

        // 2. Custom Descending Sorting
        List<String> sortedDesc = fruits.stream()
            .sorted(Comparator.reverseOrder())
            .collect(Collectors.toList());

        System.out.println("Ascending Order: " + sortedAsc);
        System.out.println("Descending Order: " + sortedDesc);
    }
}

```

• **Expected Output:**

```

Ascending Order: [Apple, Banana, Cherry, Date, Mango]
Descending Order: [Mango, Date, Cherry, Banana, Apple]

```

Problem 5: Print Collection Using forEach() (24.13.5)

```

import java.util.Arrays;
import java.util.List;

public class StreamForEachDemo {
    public static void main(String[] args) {
        List<String> frameworks = Arrays.asList("Spring Boot", "Hibernate", "React", "Docker");

        System.out.println("--- Framework List ---");
        // Traversal using forEach() with Method Reference
        frameworks.forEach(System.out::println);
    }
}

```

```
}
}
```

• **Expected Output:**

```
--- Framework List ---
Spring Boot
Hibernate
React
Docker
```

Problem 6: Collect Filtered Results into List (24.13.6)

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class StreamCollectDemo {
    public static void main(String[] args) {
        List<String> techStack = Arrays.asList("Java", "JavaScript", "Python", "JDBC", "JSON",
        , "C++");

        // Filter technologies starting with 'J' and collect into a List
        List<String> jTech = techStack.stream()
            .filter(tech -> tech.startsWith("J"))
            .collect(Collectors.toList());

        System.out.println("Technologies starting with 'J': " + jTech);
    }
}
```

• **Expected Output:**

```
Technologies starting with 'J': [Java, JavaScript, JDBC, JSON]
```

BOOK 6 END-OF-BOOK REVISION & INTERVIEW PREPARATION PACK

1. COMPLETE BOOK 6 REVISION SHEET

- **Byte Stream vs Character Stream:** Byte streams (`InputStream`/`OutputStream`) handle 8-bit bytes (binary data); Character streams (`Reader`/`Writer`) handle 16-bit Unicode characters (text files).
- **Buffered Character Streams:** `BufferedReader` and `BufferedWriter` reduce disk access by using an internal memory buffer (where the buffer size is implementation/API dependent). `BufferedReader.readLine()` reads full text lines until returning `null`.
- **Serialization:** Converting an object to a byte stream via `ObjectOutputStream`. Requires implementing `Serializable`. transient instance fields are omitted during default serialization and receive default primitive or null values upon normal deserialization unless custom logic restores them.

- **Lambda Expressions:** Anonymous functions satisfying a Single Abstract Method (‘SAM’). Local variables captured inside lambdas must be ‘final’ or effectively final.
 - **Core Functional Interfaces:** ‘Predicate<T>’ (‘test()’), ‘Function<T,R>’ (‘apply()’), ‘Consumer<T>’ (‘accept()’), ‘Supplier<T>’ (‘get()’).
 - **Stream API Lifecycle:** Source $\rightarrow$ Intermediate Operations (Lazy: ‘filter’, ‘map’, ‘sorted’) $\rightarrow$ Terminal Operation (Eager execution: ‘forEach’, ‘collect’). Streams do NOT store data and cannot be reused after terminal consumption.
 - **‘Optional<T>’:** Container object representing a value that may be present or empty, making value absence handling explicit across APIs.
 - **‘java.time’ Date & Time API:** Core classes like ‘LocalDate’, ‘LocalTime’, ‘LocalDateTime’, and ‘DateTimeFormatter’ are immutable and thread-safe, replacing legacy mutable ‘Date’ and ‘Calendar’.
-

2. IMPORTANT FRESHER INTERVIEW QUESTIONS

1. What is the difference between Byte Streams and Character Streams in Java I/O?

- **Answer:** Byte streams (‘InputStream’ / ‘OutputStream’) process data byte-by-byte (8 bits) and are suitable for binary files like images, audio, and executable code. Character streams (‘Reader’ / ‘Writer’) process data character-by-character (16-bit Unicode) and handle character encoding according to the constructor/API and Java version (with explicit ‘Charset’ support when required), making them ideal for plain text files.

2. Why should we use ‘BufferedReader’ instead of ‘FileReader’ for reading large text files?

- **Answer:** ‘FileReader’ reads characters directly from disk one by one, resulting in frequent disk I/O operations and poor performance. ‘BufferedReader’ reads a block of characters into an internal memory buffer (the exact size is implementation/API dependent, often defaulting to 8KB), dramatically reducing disk reads. Additionally, ‘BufferedReader’ provides the convenient ‘readLine()’ method.

3. What is the ‘transient’ keyword in Java Serialization?

- **Answer:** The ‘transient’ keyword is applied to instance fields to indicate that they should not be serialized by Java's default serialization mechanism. After normal deserialization, ‘transient’ fields receive default primitive or null values unless custom logic (such as custom ‘readObject()’) restores them.

4. What is a Functional Interface and what happens if we add a second abstract method to an interface annotated with ‘@FunctionalInterface’?

- **Answer:** A Functional Interface is an interface that contains exactly one abstract method (SAM). If a second abstract method is added to an interface annotated with ‘@FunctionalInterface’, the Java compiler raises a compilation error.

5. What is the difference between ‘Intermediate’ and ‘Terminal’ operations in the Stream API?

- **Answer:** Intermediate operations (`filter()`, `map()`, `sorted()`) return a new stream and are **lazy**; they do not perform processing until a terminal operation is called. Terminal operations (`forEach()`, `collect()`, `count()`) trigger stream execution and produce a non-stream result, consuming the stream.
-

3. IMPORTANT OUTPUT-PREDICTION QUESTIONS

Question 1: Lambda Variable Scoping

```
public class LambdaScopeDemo {
    public static void main(String[] args) {
        int factor = 2;
        java.util.function.Function<Integer, Integer> multiplier = num -> num * factor;
        // factor = 5; // Compilation Error if uncommented
        System.out.println(multiplier.apply(10));
    }
}
```

- **Output:** `20`
 - **Explanation:** `factor` is effectively final and can be captured by the lambda expression. Modifying `factor` after lambda definition would cause a compilation error.
-

Question 2: Reusing a Stream

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Stream;

public class StreamReuseDemo {
    public static void main(String[] args) {
        List<String> list = Arrays.asList("A", "B", "C");
        Stream<String> stream = list.stream();

        stream.forEach(System.out::print);

        try {
            long count = stream.count(); // Attempting to reuse stream
            System.out.println(" Count: " + count);
        } catch (IllegalStateException e) {
            System.out.println(" Exception: Stream already closed!");
        }
    }
}
```

- **Output:** `ABC Exception: Stream already closed!`
 - **Explanation:** Once a terminal operation (`forEach`) is executed on a stream, the stream is consumed and closed. Subsequent attempts to invoke terminal operations throw `IllegalStateException`.
-

4. DEBUGGING PRACTICE & COMMON MISTAKES

1. Forgetting to Flush or Close File Output Streams

- **Bug:** Writing text using `BufferedWriter` or `FileWriter` without calling `close()` or `flush()`, resulting in an empty or partially written output file because data remains buffered in memory. `flush()` writes buffered output to the underlying destination, whereas `close()` closes the writer and performs the required flushing.
- **Fix:** Always wrap I/O streams in a **try-with-resources** block, which automatically calls `close()` (performing the necessary flushing) upon block exit.

2. Modifying Captured Local Variables in Lambda

- **Bug:** Attempting to update a local counter variable inside a Lambda expression (`list.forEach(item -> count++)`).
- **Fix:** Local variables captured by Lambdas must be `final` or effectively final. Use atomic classes like `AtomicInteger` or use Stream reductions (`count()`, `sum()`) instead.

5. FRESHER-LEVEL MOCK INTERVIEW SCENARIO

Interviewer: Welcome! Can you explain how the Java 8 Stream API improves code readability compared to traditional `for` loops?

Candidate: Thank you, sir! Traditional `for` loops specify **HOW** to iterate over collections imperative-style, mixing loop counters, conditional checks, and mutation logic. The Stream API uses a declarative approach specifying **WHAT** data transformations are needed. Operations like filtering, mapping, and collecting are chained in a readable pipeline without mutating the source collection.

Interviewer: Excellent. How does `Optional` help handle potential value absence in Java 8?

Candidate: `Optional` acts as a container that represents a value that may be present or empty, making value absence handling explicit. Instead of returning raw `null` references from methods, returning `Optional<T>` requires the caller to explicitly handle absence using methods like `.orElse("default")` or `.ifPresent(...)`. Note that passing `null` to `Optional.of(null)` throws a `NullPointerException`, whereas `Optional.ofNullable(null)` safely yields an empty `Optional`.

6. CONCEPT COMPARISON TABLES

Feature	<code>FileReader</code> / <code>FileWriter</code>	<code>BufferedReader</code> / <code>BufferedWriter</code>
I/O Mechanism	Unbuffered (direct disk access for every char)	Buffered (uses an internal memory buffer; size is implementation/API dependent)
Performance	Slower for multi-line text operations	Fast & highly efficient
Line Operations	Reads/writes character-by-character	<code>readLine()</code> and <code>newLine()</code> support

--	--	--

Feature	Collections Framework	Stream API

-
- | **Primary Focus** | Storing & organizing data in memory | Computing & processing data in pipelines |
 - | **Memory Storage** | Holds elements in heap memory structures | Does NOT store elements |
 - | **Mutability** | Can add, remove, and modify elements | Stream operations generally do not modify the source collection; produce new results |
 - | **Reusability** | Iterable multiple times | Single-use; consumed after terminal operation |
-

7. COMPLETE BOOK 6 SYLLABUS COVERAGE CHECKLIST

- [x] **Section 23: Basic Java I/O (23.1 to 23.6.5)**
 - [x] File Handling Concept, File Class
 - [x] FileReader, FileWriter, BufferedReader, BufferedWriter
 - [x] Basic Serialization Concept (`Serializable`, `transient`)
 - [x] All 5 I/O Coding Problems solved (23.6.1 to 23.6.5)
- [x] **Section 24: Basic Java 8 Features (24.1 to 24.13.6)**
 - [x] Lambda Expression & Syntax
 - [x] Functional Interface & `@FunctionalInterface`
 - [x] Method Reference (`::`)
 - [x] Basic Stream API, `filter()`, `map()`, `forEach()`, `collect()`, `sorted()`
 - [x] Optional — Basic Awareness
 - [x] Date & Time API — Basic Awareness (`java.time`)
 - [x] All 6 Java 8 Coding Problems solved (24.13.1 to 24.13.6)